

Reflection Template

Use Case	UC-7
Use Case Name	View Task
Requirement IDs	FR8, FR20, FR22, FR23
Matching Feature Comparison Sets	CS-17
Implementation Order	Java first -> Kotlin second

Feature Relevance

Feature/s:

Extension functions

Did the feature/s occur naturally in this/these context/s?

Yes, both utility approaches were reused without any changes, no new features were introduced in this use case.

Qualitative Ratings (1-5)

- 1 = very poor/very difficult
- 2 = rather poor/difficult
- 3 = neutral/moderate
- 4 = good/easy
- 5 = very good/very easy

Criteria	Java	Kotlin
Readability	5	5
Compactness of logic	4	4
Perceived implementation effort	5	5

Justifications

Readability:

Both equally readable. The logic is minimal, fetch by id, map to response, return. No meaningful difference here.

Compactness of logic:

Both are equally compact. The only consistent difference remains the call syntax `RepositoryUtils.findByIdOrThrow(taskRepository, id, "Task")` vs. `taskRepository.findByIdOrThrow(id, "Task")`.

Implementation effort:

Both felt easy, this is the same pattern as UC-2 and UC-4.

Observed structural differences

No new structural differences compared to UC2 and UC4. This use case confirms the existing pattern.

The call syntax difference between RepositoryUtils and the extension function remains consistent, Java passes the repository as a parameter, Kotlin calls the method on it.

Final comparative summary

UC7 is a short use case that confirms existing patterns without introducing new ones. The extension function comparison is now well established across UC2, UC4, and UC7.